



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Interação em Ambientes Virtuais
Relatório: Projeto FInal

Turma: 61D

Nome: Miguel Alcobia Número: 50746

Nome: Tomás Salvador Número: 50766

Professor: Diogo Cabral

Data: 28 de Junho de 2025

Índice

Lista de Figuras	3
Introdução	4
Goblin Gamble	5
Criação do mundo Minecraft (TP1)	5
Casino e Apostas com Agentes (TP2)	7
Interação com OSC (TP3)	8
Interface do jogo	9
Conclusão	11
Bibliografia	12

Lista de Figuras

Figura 1 - Mundo do jogo	6
Figura 2 - Interior da Cabana	6
Figura 3 - Esquema que ilustra a tarefa com as suas observações/ações	7
Figura 4 - Robôs na arena	8
Figura 5 - Goblin no Casino	9
Figura 6 - Ecrã de vitória na arena	9
Figura 7 - Ecrã do Menu Inicial	10

Introdução

Ao longo do semestre, diversos trabalhos práticos foram desenvolvidos com o propósito de aplicar, de forma concreta, os conhecimentos adquiridos nas aulas. Esses projetos abordaram diversos temas, desde a geração procedimental de terrenos de forma semelhante ao que acontece em jogos do estilo do Minecraft, o treino de agentes utilizando técnicas de *machine learning*, e a integração de comandos via o protocolo OSC (Open Sound Control).

Neste trabalho final o objetivo principal era integrar e consolidar os conceitos dos três trabalhos práticos num único projeto.

E foi dessa forma que surgiu o ***Goblin Gamble***, um jogo que combina elementos dos três projetos desenvolvidos. Neste jogo, o utilizador explora um mundo gerado procedimentalmente, onde pode apanhar ouro, ao controlar o player através da *app* OSC Controler, no telemóvel. Além disso, o jogador pode apostar o ouro acumulado no desempenho de agentes de inteligência artificial treinados.

O jogo funciona da seguinte forma: Começamos numa zona gelada, ao pé de uma cabana. A cabana serve como “portal” para o casino, cujo taxa de entrada é igual a 3 barras de ouro. Desta forma, o jogador deve explorar o mapa para conseguir reunir barras de ouro suficientes para entrar no casino.

Ao chegar ao casino, é recebido por um *goblin* que explica que o jogador deve apostar na cor vermelha ou azul, gastando 3 barras de ouro. Cada cor representa um robô, que estará numa arena a competir para ser o primeiro a reunir 20 parafusos.

Caso o jogador ganhe a aposta efetuada, ganha o dobro do que apostou e caso perca, perde o valor que meteu na aposta.

Quando o jogador ficar sem ouro, pode sair do casino e repetir o processo.

Goblin Gamble

Criação do mundo Minecraft (TP1)

A primeira etapa do projeto focou-se na criação de um mundo 3D composto por blocos, à semelhança do jogo Minecraft. Inicialmente, no TP1, a geração de cubos completos revelou-se ineficiente, com sérios problemas de desempenho. Para resolver esta limitação, adotou-se uma geração gradual do terreno utilizando corrotinas, permitindo que os blocos fossem criados de forma assíncrona sem comprometer a fluidez do jogo.

Posteriormente, foi aplicada feita uma desconstrução dos blocos em *quads* (faces individuais), o que permitiu renderizar apenas as faces visíveis. Com o uso de *MeshRenderer*, foi possível reduzir o número de polígonos e melhorar o desempenho gráfico.

As texturas foram aplicadas através de um atlas de texturas e usaram-se coordenadas UV para mapear corretamente as diferentes faces de cada tipo de bloco (gelo, neve, ouro etc.).

Para aumentar ainda mais a eficiência, os blocos foram agrupados em *chunks* de $16 \times 16 \times 16$ blocos. Cada *chunk* foi tratado como uma única *mesh* e, desta forma, reduziu-se o número de chamadas para a renderização do mundo. Foram também implementadas verificações para eliminar as faces internas, para garantir que apenas as superfícies visíveis fossem renderizadas.

Para modelar o relevo, utilizou-se o algoritmo Perlin Noise, que gera paisagens suaves e naturais com variações de altura mais realistas para que não haja mudanças abruptas no relevo do mundo.

A geração do mundo foi também feita em tempo real, com a criação e a remoção automática dos *chunks* consoante a posição do jogador, através de um algoritmo baseado no Flood Fill. Desta forma, mantiveram-se ativos apenas os *chunks* mais próximos do jogador.

Por fim, foi introduzida a interação do jogador com o ambiente, permitindo modificar o terreno em tempo real, tal como no Minecraft. Os blocos podem ser colocados ou removidos, com atualização automática das malhas envolvidas para manter a performance e a consistência visual.

Após a geração do mapa foram implementadas alguns elementos importantes para o funcionamento do jogo como a geração de blocos de ouro, o player poder apanhá-los e a criação de uma casa no ponto de *spawn*, que é o local onde o jogador pode entrar no casino para poder apostar o ouro que apanhou pelo mapa.

Durante o processo de criação dos blocos em cada *chunk*, foi então adicionada a lógica para gerar blocos de ouro de forma aleatória e controlada. A geração acontece no momento em que cada posição dentro da *chunk* é analisada para decidir qual tipo de bloco será colocado ali. Se for detetado que a posição está logo acima da camada de neve (para garantir que são gerados na superfície, e ficam visíveis para o jogador), existe uma probabilidade pequena de que um bloco de ouro seja colocado nesse local. Esta chance é avaliada individualmente para cada posição dentro da *chunk*, para permitir que os blocos de ouro surjam de forma espalhada pelo mundo.

Em relação à preservação da casa e do seu *chunk*, foi necessário implementar um mecanismo para impedir que o *chunk* onde a casa foi construída seja destruída automaticamente. O sistema do mundo foi feito de modo a remover dinamicamente os *chunks* que estão muito distantes do jogador, com o objetivo de melhorar o

desempenho do jogo. Para contornar isso e garantir que a casa esteja sempre no mundo, logo no início da execução, a posição do *chunk* onde a casa foi instanciada é registada numa variável específica. Posteriormente, sempre que o sistema faz a verificação para remover *chunks* fora do alcance definido, essa variável é consultada e impede que a *chunk* correspondente à casa seja incluída na lista de chunks que serão removidas.

Desta forma, mesmo que o jogador se afaste, a chunk da casa não é afetada pelo sistema de limpeza automática, e todos os objetos dentro dela continuam presentes e funcionais no mundo.

Para que o jogador não se perca no mundo durante a procura de ouro foi criada uma bússola com o objetivo de indicar a direção da casa. No código, a bússola é atualizada a cada *frame* com base na posição e direção da câmara, calculando o ângulo entre o olhar do jogador e o destino do marcador da casa. Esse ângulo é convertido numa posição horizontal dentro da barra da bússola, garantindo que os ícones se movam de forma precisa e intuitiva consoante o jogador muda de direção.



Figura 1 - Mundo do jogo

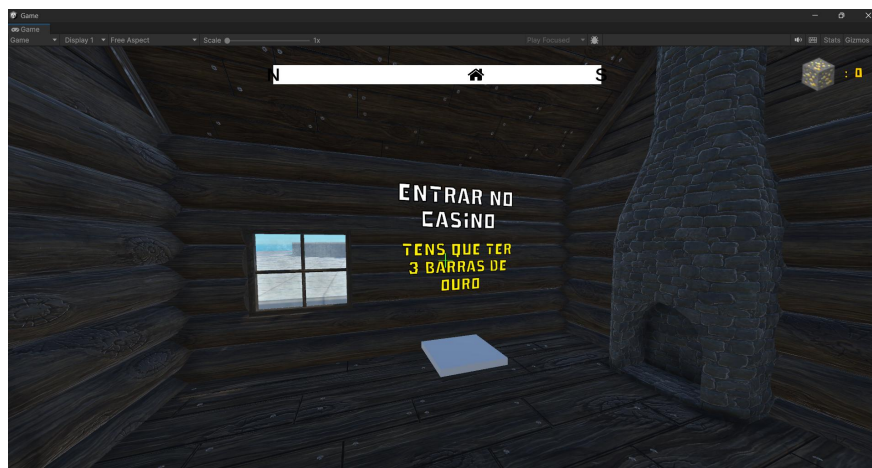


Figura 2 - Interior da Cabana

Casino e Apostas com Agentes (TP2)

A parte principal do casino são os robôs nos quais o jogador pode apostar. Nesta fase o jogador vai ver uma arena onde dois robôs, um vermelho e um azul, apanham parafusos do chão. Estes robôs representam os agentes que foram estudados e implementado durante o semestre.

A aposta funciona da seguinte forma: o jogador aposta 3 barras de ouro para tentar adivinhar qual dos robôs apanhará primeiro 20 parafusos. Se o jogador ganhar a apostas ganha 6 barras de ouro (duplica o que apostou) e caso perca, o jogador fica sem o valor que apostou.

O funcionamento dos ML-Agents no trabalho final é semelhante ao seu funcionamento no TP2. Neste projeto o agente foi substituído por um robô, a comida por parafusos e o botão por uma caixa de ferramentas. Neste novo cenário, o robô tem de executar uma sequência de ações: primeiro, deslocar-se até à caixa de ferramentas, e depois dirigir-se aos parafusos que são criados após a interação com a caixa.

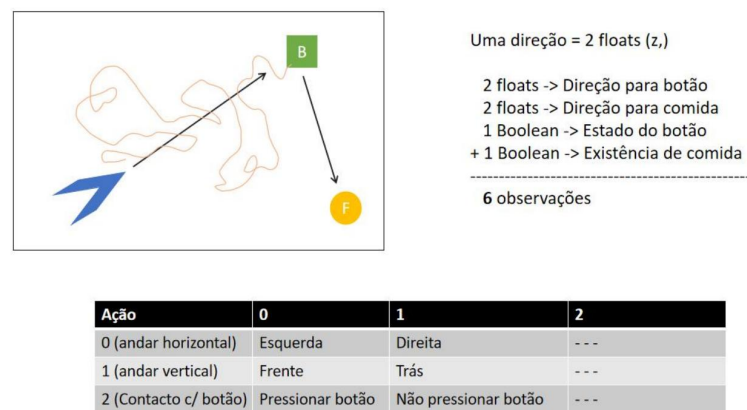


Figura 3 - Esquema que ilustra a tarefa com as suas observações/ações

As observações do robô incluíam a direção relativa até à caixa de ferramentas (em X e Z), se a caixa já tinha sido ativada, se os parafusos estavam presentes no ambiente e, caso estivessem, a direção até aos mesmos. De forma a garantir que todos os elementos do ambiente reagissem corretamente a cada reinício de episódio ou interação, foi utilizada a classe `EventHandler`, que permitia notificar todos os objetos relevantes sobre alterações de estado, como o início de um novo episódio ou a ativação da caixa.

As recompensas foram ajustadas consoante o novo contexto: o robô era recompensado positivamente ao alcançar a caixa de ferramentas e novamente ao recolher os parafusos. Em contrapartida, era penalizado sempre que colidia com as paredes do cenário. Para aumentar a variabilidade e promover um comportamento mais generalizado, a posição da caixa de ferramentas é feita de forma aleatória a cada novo episódio, à semelhança do que acontecia no TP2.

O treino do robô iniciou-se com recompensas extrínsecas, ou seja, recompensas fornecidas diretamente pelo ambiente com base nas ações realizada, usando parâmetros como `gamma`, que controla a importância de recompensas futuras, e `strength`.

Para otimizar o tempo de treino, criaram-se múltiplas instâncias do cenário (16 no total), o que permitiu que vários robôs aprendessem em paralelo. O resultado final foi um robô capaz de completar a tarefa com elevado desempenho, tendo-se registado uma recompensa média de 1.7, muito próxima da média de 1.8 obtida com as

demonstrações humanas, feitas no modo Heurístico.

Interação com OSC (TP3)

Para integrar o TP3, foi integrado o protocolo OSC, um protocolo de comunicação moderno, flexível e de baixa latência, amplamente utilizado em contextos interativos. Este protocolo permite a troca de mensagens entre aplicações e dispositivos através de redes locais, tornando-o ideal para experiências em tempo real e controlos remotos personalizados.

A sua utilização neste projeto teve como principal objetivo estabelecer uma ponte de comunicação entre o Unity e um dispositivo móvel, permitindo ao jogador controlar ações do jogo através de interfaces criadas no telemóvel. Para isso, recorreu-se à biblioteca *extOSC*, uma ferramenta fácil de integrar com o Unity, que fornece todas as funcionalidades necessárias para enviar e receber mensagens OSC.

Criou-se um sistema de controlo de movimento do jogador com um *joystick* virtual no dispositivo móvel, através da aplicação OSC Controller. As coordenadas do *joystick* são enviadas como mensagens OSC para os endereços `/osc/joystickX` e `/osc/joystickY`, representando o eixo horizontal e vertical, respetivamente. O *script* `PlayerController` interpreta esses valores em tempo real, normaliza-os, e aplica um vetor de movimento à personagem controlada no jogo. O valor do *joystick* parte de um ponto neutro (0.5, 0.5) e é transformado numa gama entre -1 e 1 para representar a direção do movimento.

Para evitar movimentos involuntários devido a pequenas variações nos valores recebidos, foi definida uma zona morta (*deadzone*), ignorando valores cujo vetor tenha uma magnitude inferior a 0.25.

Foi também desenvolvido um sistema de controlo remoto para a parte do casino, onde o jogador pode apostar em dois robôs distintos (vermelho ou azul). Através da aplicação móvel, configuraram-se dois botões virtuais que enviam mensagens OSC para os endereços `/osc/red` e `/osc/blue`. Do lado do Unity, foi criado o *script* `CasinoUI`, que inicializa um receptor OSC na porta 7001 e associa essas mensagens a dois métodos (`OnReceiveRed` e `OnReceiveBlue`). Estes métodos acionam as respetivas apostas no jogo, substituindo o clique direto no botão da interface gráfica do Unity. Desta forma o utilizador pode apostar através o seu telemóvel.

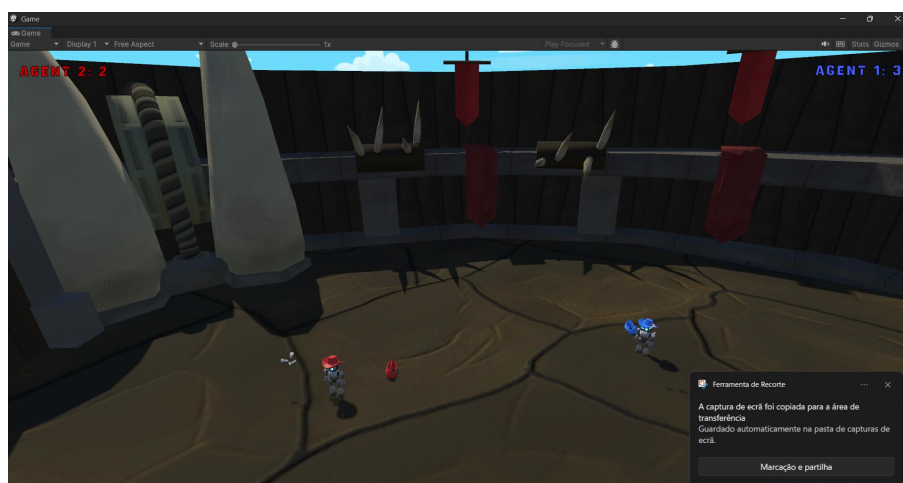


Figura 4 - Robôs na arena

Interface do jogo

Para melhorar a experiência do utilizador foi criado um menu inicial através de um Canvas, com 3 botões. Um para dar início ao jogo, outro para abrir um menu de opções, onde pode ser controlado o volume do som do jogo, e outro para sair.

Relativamente ao casino, foi criado um cenário para representar o balcão de entrada onde aparecem dois botões que permitem escolher a cor do robô na qual o jogador quer apostar. Para melhorar a experiência do utilizador durante todo o jogo aparece no canto superior direito do ecrã o número de barras de ouro que este possui, de forma a manter o jogador sempre informado.

Durante o funcionamento dos agentes, também aparece um contador para cada agente que demonstra o número de parafusos que cada robô já apanhou, e quando o primeiro robô alcança os 20 parafusos, no centro do ecrã aparece a informação se o jogador ganhou ou perdeu a aposta. A acompanhar essa mensagem também se pode ouvir um som que corresponde à vitória ou à derrota, consoante o resultado da aposta.



Figura 5 - Goblin no Casino

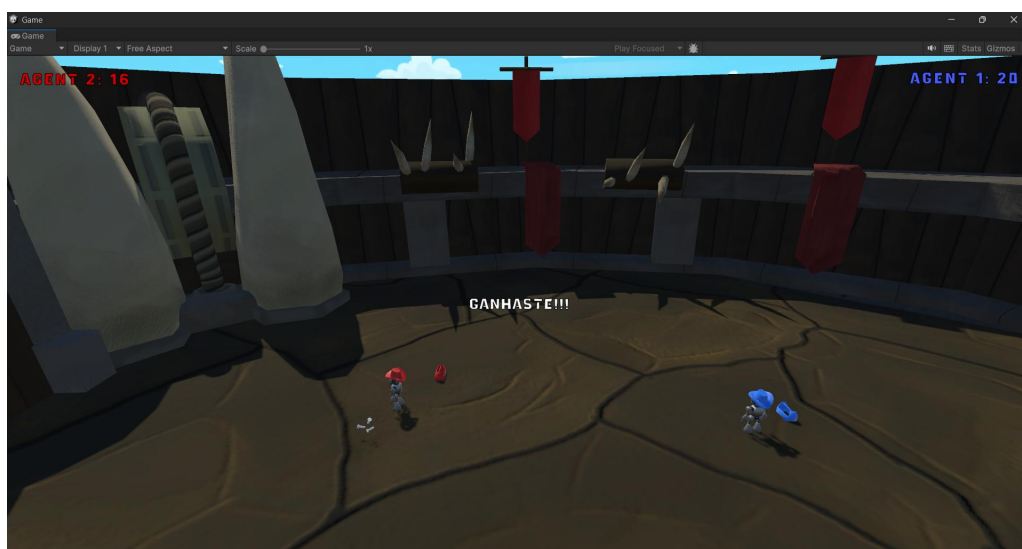


Figura 6 - Ecrã de vitória na arena

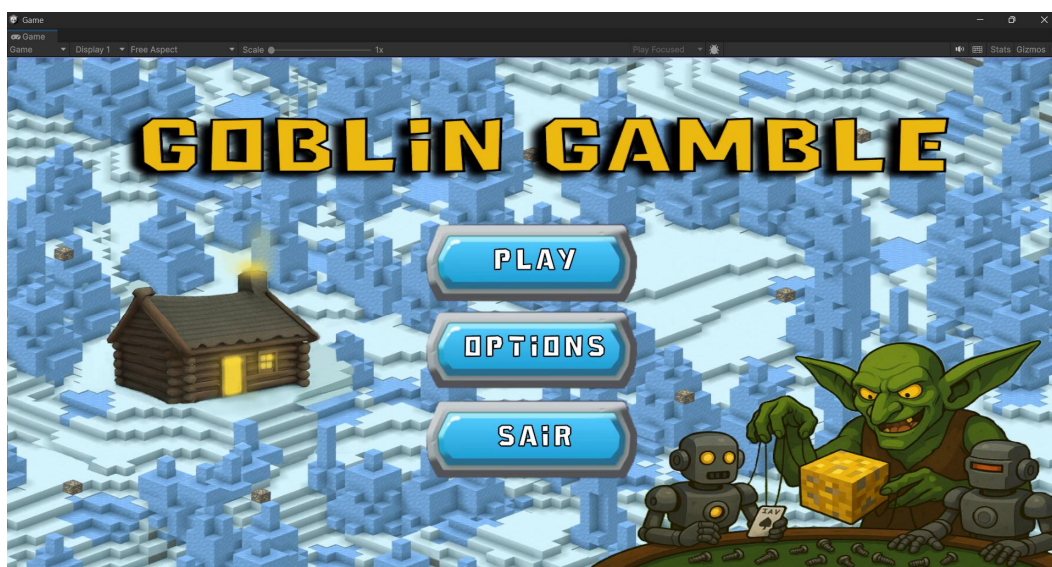


Figura 7 - Ecrã do Menu Inicial

Conclusão

Este projeto permitiu consolidar os conhecimentos adquiridos ao longo do semestre, integrando os conceitos dos três trabalhos práticos num jogo. A combinação da geração procedimental de mundos (TP1), a utilização de agentes de *machine learning* (TP2) e a interação via protocolo OSC (TP3) resultou numa experiência interativa, onde o jogador explora um ambiente dinâmico, recolhe recursos e pode fazer apostas no desempenho de agentes treinados.

A maior dificuldade encontrada foi encontrar um papel adequado para a integração do TP2 (ML Agents) no projeto final. Inicialmente, considerámos utilizar o terreno irregular do mundo Minecraft para os treinos dos agentes, mas percebemos que a superfície irregular dificultava o processo de aprendizagem. Além disso, faltavam-nos ideias claras sobre como encaixar os agentes de forma natural na narrativa do jogo. Como solução, optámos por adaptar o TP2, criando uma arena no casino, onde os robôs competem para recolher parafusos.

Outro desafio significativo foi a implementação do protocolo OSC, que exigiu ajustes na configuração da *firewall* para garantir a comunicação entre o Unity e o telemóvel. Apesar das dificuldades, a integração do OSC permitiu um controlo intuitivo e remoto do jogo, melhorando a experiência do utilizador.

Em suma, o grupo teve bastante gosto em desenvolver este projeto e a solução encontrada para o TP2 destacou como limitações técnicas podem ser transformadas em novas ideias.

Bibliografia

Abrantes, A. (2025). Vídeos Unidade Curricular de Interação em Ambientes Virtuais. via Moodle

6 Minute PAUSE MENU Unity Tutorial. (n.d.). www.youtube.com.

<https://www.youtube.com/watch?v=9dYDBomQpBQ>

Keyiter. (2021, November 6). *How to make a compass bar | Unity in 60 seconds.* YouTube.

<https://www.youtube.com/watch?v=XcpTC1VYVNE>

Desenvolvedor Unity. (2022, June 10). *Aprenda como criar um MENU PRINCIPAL na Unity de forma Simples e Elegante.* YouTube. <https://www.youtube.com/watch?v=yIaAE9bLxXI>